

Linux User Permissions Project

Commands, Purpose & Terminal Screenshots — Beginning to End

Project Path	/root/Project
Roles	developer, tester, deployer
Tools Used	mkdir, touch, useradd, id, chown, chmod, setfacl, getfacl, ls, tree
Access Model	Ownership + 770 base permissions + ACL (rwx) between role users

This document walks through a complete Linux user-permissions project performed in /root/Project. It covers creating role-based directories and files, creating and verifying Linux user accounts, assigning ownership, applying base 770 permissions, and configuring ACL/FACL entries so each role can share controlled access with the other two roles.

Part 1 — Project Structure & Directory Creation

The project uses three role-based directories — developer, tester, and deployer — each containing one file named after its role.

Purpose: Organizing work by role directory is a common real-world pattern: it keeps each team's files separate while still living under one shared project tree, making ownership and permissions easy to reason about.

Commands Used

```
mkdir -p Project/{developer,tester,deployer}
```

Purpose: Creates the Project directory and all three role subdirectories in a single command. The -p option also creates any missing parent directories.

```
touch developer/developer.txt
```

```
touch tester/tester.txt
```

```
touch deployer/deployer.txt
```

Purpose: Creates one empty placeholder file inside each role directory, representing that role's work.

Step 1.1 — Project Setup: Directory Creation & Tree Output

```
[root@server]~# cd /root
[root@server]~# pwd
/root
[root@server]~# ls
anaconda-ks.cfg  Desktop  Downloads  Music  Public  Templates  umask1
cyber            Documents  git        Pictures  ssh.conf  umask      Videos
[root@server]~# mkdir -p Project/{developer,tester,deployer}
[root@server]~# cd Project
[root@server]~/Project# touch deployer/deployer.txt
[root@server]~/Project# touch tester/tester.txt
[root@server]~/Project# touch developer/developer.txt
[root@server]~/Project# tree
├── deployer
│   └── deployer.txt
├── developer
│   └── developer.txt
└── tester
    └── tester.txt

4 directories, 3 files
[root@server]~/Project# ls -l
total 0
drwxr-xr-x. 2 root root 26 Aug 29 23:06 deployer
drwxr-xr-x. 2 root root 26 Aug 29 23:07 developer
drwxr-xr-x. 2 root root 24 Aug 29 23:07 tester
```

mkdir creates the three role directories, touch adds a file to each, and tree/ls -l confirm the resulting structure and default permissions.

Part 2 — Creating & Verifying Linux User Accounts

Three Linux user accounts were created — one per role — so that ownership and access control could later be tied to real system users rather than the generic root account.

Purpose: Real infrastructure is never managed as a single root user. Creating a dedicated account per role mirrors how companies assign least-privilege access — each person or service only gets the account (and permissions) relevant to their job.

Commands Used

<code>useradd tester_team</code>
<code>useradd deployer_team</code>
<code>useradd developer_team</code>
Purpose: Creates a new Linux user account for each role, each with its own UID, GID, and home directory.
<code>id tester_team developer_team deployer_team</code>
Purpose: Confirms each account exists and shows its UID, primary GID, and group membership.

Step 2.1 — User Creation & ID Verification

```
[root@server]~/Project# useradd tester_team
[root@server]~/Project# useradd deployer_team
[root@server]~/Project# useradd developer_team
[root@server]~/Project# id tester_team developer_team deployer_team
uid=1027(tester_team) gid=1027(tester_team) groups=1027(tester_team)
uid=1029(developer_team) gid=1029(developer_team) groups=1029(developer_team)
uid=1028(deployer_team) gid=1028(deployer_team) groups=1028(deployer_team)
```

`useradd` creates the three role accounts; `id` confirms their UID/GID and group membership — `tester_team` (1027), `deployer_team` (1028), `developer_team` (1029).

Part 3 — Assigning Ownership

Ownership of every file and directory was changed from root to the matching role account, so each team's files and folder belong to that team's own user and group.

Purpose: Ownership is the foundation permissions are built on — chmod and ACL rules only make sense once the right user and group actually own the resource. Without this step, every file would still belong to root regardless of later permission changes.

Commands Used

```
chown developer_team:developer_team developer/developer.txt
```

```
chown deployer_team:deployer_team deployer/deployer.txt
```

```
chown tester_team:tester_team tester/tester.txt
```

Purpose: Assigns each file to its matching role user and group.

```
chown deployer_team:deployer_team deployer
```

```
chown tester_team:tester_team tester
```

```
chown developer_team:developer_team developer
```

Purpose: Assigns each role directory itself (not just the file inside it) to its matching user and group.

Step 3.1 — Ownership Commands (Files)

```
[root@server]~/Project# chown devloper_team:devloper_team devloper/devloper.txt
[root@server]~/Project# chown deployer_team:deployer_team deployer/deployer.txt
[root@server]~/Project# chown tester_team:tester_team tester/tester.txt
```

chown applied to each role's file, assigning it to the matching team user and group.

Step 3.2 — Ownership Commands (Directories)

```
[root@server]~/Project# chown deployer_team:deployer_team deployer
[root@server]~/Project# chown tester_team:tester_team tester
[root@server]~/Project# chown devloper_team:devloper_team devloper
```

chown applied to each role directory itself, completing the ownership change for the full tree.

Part 4 — Verifying Ownership

A recursive listing was used to confirm that every directory and file now shows the correct owner and group after the chown commands.

Purpose: Verifying a change immediately after making it is a basic sysadmin habit — it catches mistakes (like a typo in a username) before they cause access problems later, when they'd be harder to trace back.

Commands Used

`ls -lR`

Purpose: Recursively lists permissions, owner, group, size, date, and name for every item in the project tree.

Step 4.1 — Recursive Ownership Listing

```
[root@server]~/Project# ls -lR
.:
total 0
drwxr-xr-x. 2 deployer_team deployer_team 26 Aug 29 23:06 deployer
drwxr-xr-x. 2 devloper_team devloper_team 26 Aug 29 23:07 devloper
drwxr-xr-x. 2 tester_team tester_team 24 Aug 29 23:07 tester

./deployer:
total 0
-rw-r--r--. 1 deployer_team deployer_team 0 Aug 29 23:06 deployer.txt

./devloper:
total 0
-rw-r--r--. 1 devloper_team devloper_team 0 Aug 29 23:07 devloper.txt

./tester:
total 0
-rw-r--r--. 1 tester_team tester_team 0 Aug 29 23:07 tester.txt
[root@server]~/Project#
```

`ls -lR` confirms each directory and its file are now owned by the correct role user and group (e.g., `deployer_team` owns `deployer/` and `deployer.txt`).

Part 5 — Setting Base 770 Permissions

Each role directory was recursively set to permission mode 770 — full access for the owner and group, and no access for anyone else.

Purpose: 770 is a standard 'team-private' permission pattern: the owning user and group get full read/write/execute access, while everyone else on the system is completely locked out. This is the baseline before any cross-role access is added.

Commands Used

```
chmod -R 770 developer
```

```
chmod -R 770 deployer
```

```
chmod -R 770 tester
```

Purpose: Recursively sets mode 770 on each role directory and its contents — owner = rwx (7), group = rwx (7), others = --- (0).

Step 5.1 — Applying 770 Permissions

```
[root@server]~/Project# chmod -R 770 developer  
[root@server]~/Project# chmod -R 770 deployer  
[root@server]~/Project# chmod -R 770 tester
```

chmod -R 770 applied to each of the three role directories.

Part 6 — Configuring ACL / FACL Cross-Role Access

Named-user ACL (Access Control List) entries were added so each role's other two teams could access its directory with rwx permissions, without changing the underlying ownership or the base 770 permissions.

Purpose: Standard Unix permissions only support one owner and one group — they can't express 'let these two specific other users in too.' ACLs solve exactly that problem, letting teams collaborate across role directories without loosening permissions for everyone else.

Commands Used

```
setfacl -R -m u:tester_team:rwx,u:deployer_team:rwx developer
```

```
setfacl -R -m u:tester_team:rwx,u:developer_team:rwx deployer
```

```
setfacl -R -m u:deployer_team:rwx,u:developer_team:rwx tester
```

Purpose: Recursively grants the other two role users rwx access on each role directory, on top of the existing owner/group permissions.

Step 6.1 — Setting ACL Entries

```
drwxrwx---. 2 deployer_team deployer_team 26 Aug 29 23:06 deployer
drwxrwx---. 2 developer_team developer_team 26 Aug 29 23:07 developer
drwxrwx---. 2 tester_team   tester_team   24 Aug 29 23:07 tester

./deployer:
total 0
-rwxrwx---. 1 deployer_team deployer_team 0 Aug 29 23:06 deployer.txt

./developer:
total 0
-rwxrwx---. 1 developer_team developer_team 0 Aug 29 23:07 developer.txt

./tester:
total 0
-rwxrwx---. 1 tester_team   tester_team   0 Aug 29 23:07 tester.txt
```

setfacl commands applied to each role directory, followed by a listing showing the '+' style rwxrwx--- permissions that indicate an ACL is now active.

Part 7 — Verifying ACL Entries

getfacl was used to display the full ACL entry list for each role directory and confirm the named-user permissions were applied correctly.

Purpose: ACLs are easy to get subtly wrong (e.g. granting access to the wrong user, or forgetting the mask entry). Reviewing the getfacl output for every directory confirms the access model works exactly as intended before considering the task complete.

Commands Used

```
getfacl -R developer
```

```
getfacl -R deployer
```

```
getfacl -R tester
```

Purpose: Recursively displays the ACL entries for every directory and file under each role directory.

Step 7.1 — Developer ACL Verification

```
[root@server]~/Project# sudo setfacl -R -m u:tester_team:rwX,u:deployer_team:rwX developer
[root@server]~/Project# sudo setfacl -R -m u:tester_team:rwX,u:developer_team:rwX deployer

[root@server]~/Project# sudo setfacl -R -m u:deployer_team:rwX,u:developer_team:rwX tester
```

setfacl commands re-shown for reference alongside the ACL verification pass across all three role directories.

Step 7.2 — Deployer ACL Verification

```
[root@server]~/Project# getfacl -R developer
# file: developer
# owner: developer_team
# group: developer_team
user::rwX
user:tester_team:rwX
user:deployer_team:rwX
group::rwX
mask::rwX
other::---

# file: developer/developer.txt
# owner: developer_team
# group: developer_team
user::rwX
user:tester_team:rwX
user:deployer_team:rwX
group::rwX
mask::rwX
other::---
```

getfacl -R developer output — tester_team and deployer_team both show rwX access alongside the owning developer_team user.

Step 7.3 — Tester ACL Verification


```
[root@server]~/Project# getfacl -R deployer
# file: deployer
# owner: deployer_team
# group: deployer_team
user::rwx
user:tester_team:rwx
user:developer_team:rwx
group::rwx
mask::rwx
other::---

# file: deployer/deployer.txt
# owner: deployer_team
# group: deployer_team
user::rwx
user:tester_team:rwx
user:developer_team:rwx
group::rwx
mask::rwx
other::---
```

getfacl -R deployer output — tester_team and developer_team both show rwx access alongside the owning deployer_team user.

Step 7.4 — Final Verification Pass

```
[root@server]~/Project# getfacl -R tester
# file: tester
# owner: tester_team
# group: tester_team
user::rwx
user:deployer_team:rwx
user:developer_team:rwx
group::rwx
mask::rwx
other::---

# file: tester/tester.txt
# owner: tester_team
# group: tester_team
user::rwx
user:deployer_team:rwx
user:developer_team:rwx
group::rwx
mask::rwx
other::---
```

getfacl -R tester output — deployer_team and developer_team both show rwx access alongside the owning tester_team user, completing the cross-role access model.

Summary

This project demonstrated a complete Linux access-control workflow: building a role-based directory structure, creating dedicated user accounts per role, assigning correct ownership, applying a secure 770 baseline, and layering ACL entries to allow controlled cross-role collaboration — all verified step by step with ls, id, and getfacl.